

# Informe Tarea 1

Matías Ardissoni Otazo, [mardissoni@usm.cl](mailto:mardissoni@usm.cl)

Miguel Candia Lorca, [mcandial@usm.cl](mailto:mcandial@usm.cl)

Sebastian Ibaceta Ramos, [sibaceta@usm.cl](mailto:sibaceta@usm.cl)

Joshua Leal Guerra, [jlealg@usm.cl](mailto:jlealg@usm.cl)

Agustín Valenzuela Torrealba, [avalenzuelat@usm.cl](mailto:avalenzuelat@usm.cl)

## 1. Parte 1

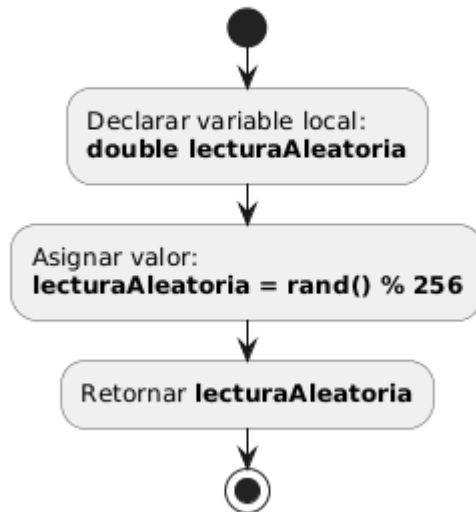
### 1.1. Diseño de la función lecturaSensor()

Primeramente, la función lecturaSensor() es una función de tipo double por lo que entregará un punto flotante de precisión doble, además de no necesitar ninguna variable de entrada. Se menciona con anterioridad que la funcionalidad de esta es simular la lectura de un sensor. Al entrar en la función encontramos la declaración de una variable local de tipo de dato double llamada "lecturaAleatoria", posteriormente a esta variable se le entrega un valor dado  $\text{rand()} \% 256$ , rand() es una función que entrega un número aleatorio,  $\% 256$  nos indica que el valor obtenido por rand será dividido por 256 y el residuo será el valor asignado a la variable.

La utilización de Acción como un enum es principalmente para no caer en la redundancia, sabemos que un sensor consta de tres fases, una donde se inicializa, otra donde se realizan las mediciones y finalmente una donde entrega los resultados. El tener estos 3 modos en un enum, permite crear una sola función que sea utilizable para todos los casos, permitiendo simplificar la función main

Diagrama de flujo del diseño de la función lecturaSensor():

### Diseño de la función lecturaSensor()

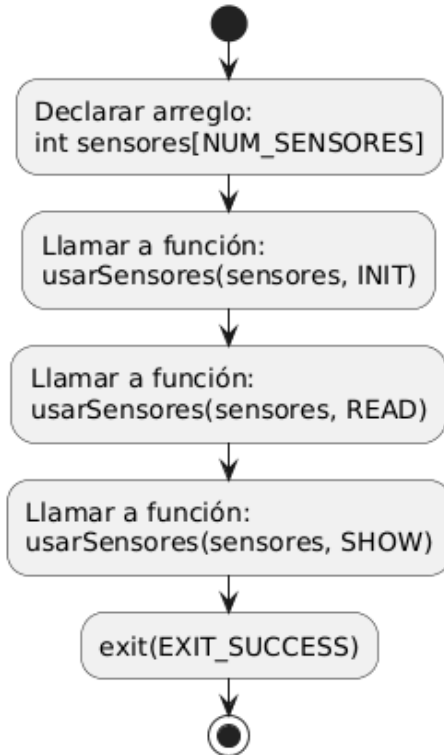


### 1.2. Diseño completo de la nueva función main()

En el nuevo diseño, la función `main()` actúa como un controlador de alto nivel. En lugar de iterar directamente sobre el arreglo de sensores, llama a la función `usarSensores()` pasando como argumentos el puntero al arreglo y una constante simbólica (`INIT`, `READ` o `SHOW`). Esto permite una arquitectura modular, donde el `main` solo gestiona el flujo de estados del sistema, mientras que la lógica de implementación queda encapsulada en la función de interfaz.

diagrama de flujo de la función `main()`:

### Diagrama de Flujo: Función main()



#### 1.2. Diseño completo de la nueva función main()

La nueva versión de nuestra función main es el resultado de lo planteado con anterioridad. Primeramente, la función entrega el tipo de int, además de recibir las variables int argc y char\* argv que guardan información dada desde la consola de comandos. Ya ingresando dentro de la función se declara un arreglo de tipo int llamado sensores con una extensión dada por la constante NUM\_SENSORES, posteriormente es llamada la función usarSensores entregando como variables nuestro arreglo sensores e INIT.

Ingresando en la función usarSensores, al ser de tipo void no devuelve ningún resultado, además recibe como variables un arreglo de tipo int y una acc de nuestro enum. switch(acc) tiene la función de buscar el valor introducido en los diferentes casos, como en primera instancia se colocó INIT, comenzará el ciclo for que recorre todo el arreglo sensores ingresando a cada índice el valor 0, al terminar break le dice al programa que tiene que terminar de ejecutar el bloque correspondiente a switch

Regresando a la función main, lo siguiente será ejecutar de nuevo la función usarSensores pero esta vez cambiando el INIT por READ. Similar a lo que ocurrió anteriormente se entra dentro de switch, pero esta vez se utiliza el código asociado a READ. Se inicia un ciclo for el cual recorre todo el arreglo, además de introducir a cada índice del arreglo un valor dado por la función lecturaSensor. Finalmente, al terminar el ciclo se produce un break que nos devuelve a la función main

Otra vez en la función main se vuelve a llamar a la función usarSensores, pero esta vez utilizamos SHOW como variable acc, esto es utilizado por el switch para iniciar un ciclo for que recorre el arreglo y por cada índice imprime: "Sensor (Número del índice): (Valor del índice) [mV]", al terminar el ciclo nos devuelve al main por medio de un break.

Finalmente, el programa da una finalización mediante la instrucción exit(EXIT\_SUCCESS).

## **2. Parte 2**

### **2.1. Hito 1**

¿Qué es lo que retorna la función clock()?

Clock es una función de la librería <time.h> que tiene como función devolver el tiempo del CPU desde que el programa inició

¿Qué unidades tienen las variables inicio y fin?

Las unidades utilizadas para las variables inicio y fin son los ciclos de reloj del CPU

¿Por qué para obtener el tiempo total de duración del ciclo for se debe dividir la diferencia entre fin e inicio con el valor constante CLOCKS\_PER\_SEC?

Como el valor retornado no son segundos como tal hay que realizar una conversión de unidades, en este caso se consigue con la diferencia entre el final y el inicio y la división de una constante. La constante nos indica la cantidad de ticks por segundo son entregados por el procesador del sistema operativo

Explicar qué hace (double) como prefijo de (fin-inicio)

El double funciona como una conversión explícita, debido a que tanto fin como inicio están declarados como enteros, entonces al realizar la división puede existir una pérdida de datos.

## 2.2. Hito 2

Primeramente como nuestra función entrega un promedio, la función será de tipo punto flotante de precisión doble, además recibirá como variable un enumerador de nuestro enum caso\_t. Ya entrando al código de la función establecemos las variables locales de tipo double tiempoPromedio, cantidadIteraciones, tiempoTotal y dos variables de tipo uint64\_t inicio y fin.

Le damos a tiempoPromedio un valor de 0.0, a cantidadIteraciones un valor de 1e8 e inicio le establecemos un valor de clock(). Después de establecer las variables iniciamos un ciclo for que termina al completar la misma cantidad de vueltas que cantidadIteraciones, dentro del ciclo hay un switch que dependiendo del enumerador entregado iniciará cierto código. En caso de entregar VACIO se realizará un break de inmediato. Para ESPACIO se imprimirá en pantalla " " para posteriormente realizar un break. En el caso ASTERISCO se imprimirá "\*" y después se realiza un break. Finalmente en NEWLINE imprime "/n" . En cada uno de estos casos el ciclo se repite hasta completar las mismas vueltas que el valor cantidadIteraciones.

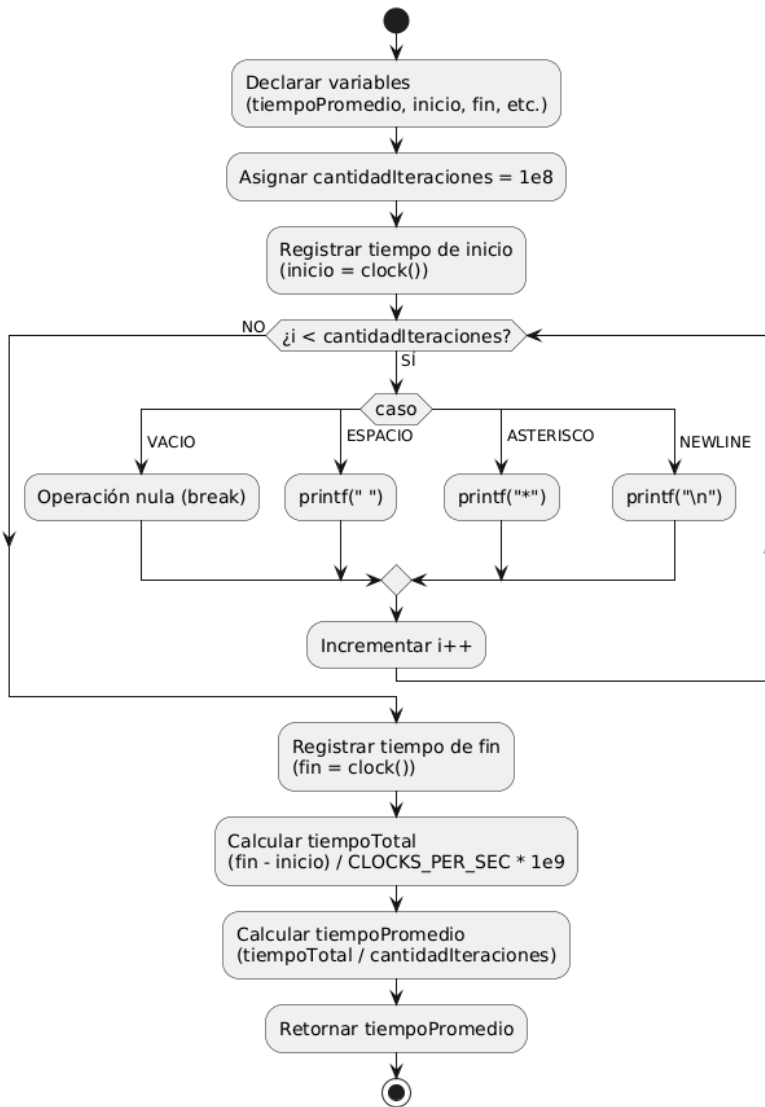
Saliendo del ciclo, se le entrega a la variable fin el valor clock(), ahora a la variable tiempo promedio se le establece como la diferencia entre fin e inicio dividido por la constante CLOCKS\_PER\_SEC multiplicado por 1e9, además de colocar el prefijo double para evitar la pérdida de datos.

Para obtener el valor de tiempoPromedio dividimos el valor de tiempoTotal en el valor de cantidadIteraciones

Finalmente, establecemos que el valor a retornar sea tiempoPromedio.

Diagrama de flujo:

**Diagrama de Flujo: Función cicloPrueba(caso)**



### 2.3. Hito 3

Para la medición de los tiempos de ejecución se utilizó la librería estándar de C `<time.h>`. El método consistió en utilizar la función `clock()` para registrar el número de ticks del procesador justo antes de iniciar el ciclo `for` y nuevamente al finalizarlo. El tiempo total se calculó restando ambos valores, realizando un cast a `double` para mantener la precisión decimal, y dividiendo el resultado por la constante del sistema `CLOCKS_PER_SEC`. Finalmente, este valor en segundos se multiplicó por  $10^9$  para convertirlo a

nanosegundos y se dividió por la cantidad de iteraciones del ciclo para obtener el tiempo promedio por operación.

Se realizaron un total de 10 ejecuciones independientes del programa compilado. Además, para garantizar que la medición fuera perceptible y minimizar el impacto de procesos en segundo plano del sistema operativo, cada una de estas ejecuciones internas constó de un ciclo de 100.000.000 iteraciones por cada caso de prueba (VACIO, ESPACIO, ASTERISCO, NEWLINE).

| Caso      | tprom1 | tprom2 | tprom3 | tprom4 | tprom5 | tprom6 | tprom7 | tprom8 | tprom9 | tprom10 | Promedio | DesvStd |
|-----------|--------|--------|--------|--------|--------|--------|--------|--------|--------|---------|----------|---------|
| Nada      | 1.8    | 1.09   | 1.23   | 1.24   | 1.27   | 1.58   | 1.1    | 1.23   | 1.41   | 1.58    | 1.35     | 0.24    |
| Espacio   | 41     | 46.75  | 40.55  | 34.81  | 34.78  | 35.52  | 49.81  | 44.3   | 42.81  | 36.03   | 40.64    | 5.41    |
| Asterisco | 46.44  | 37.5   | 37.43  | 33.12  | 35.64  | 40.57  | 30.7   | 41.99  | 39.67  | 50.55   | 39.36    | 6.2     |
| NewLine   | 41.47  | 46.86  | 41.12  | 36.01  | 38.45  | 40.34  | 44.45  | 48.39  | 44.29  | 37.65   | 41.9     | 4.21    |

En la tabla además de los datos calculados, se muestra el promedio y desviación estándar de cada caso, en el caso espacio y NewLine se ve una desviación estándar alta, lo que significa que los datos tienen una alta dispersión.

Las pruebas fueron ejecutadas en un equipo con sistema operativo Windows 11, equipado con un procesador intel core i5-10300H y 16gb de ram.

El código fuente fue compilado utilizando GCC (GNU Compiler Collection) proporcionado por el entorno MSYS2. La versión exacta del compilador utilizada para ejecutar las pruebas fue la 15.2.0."